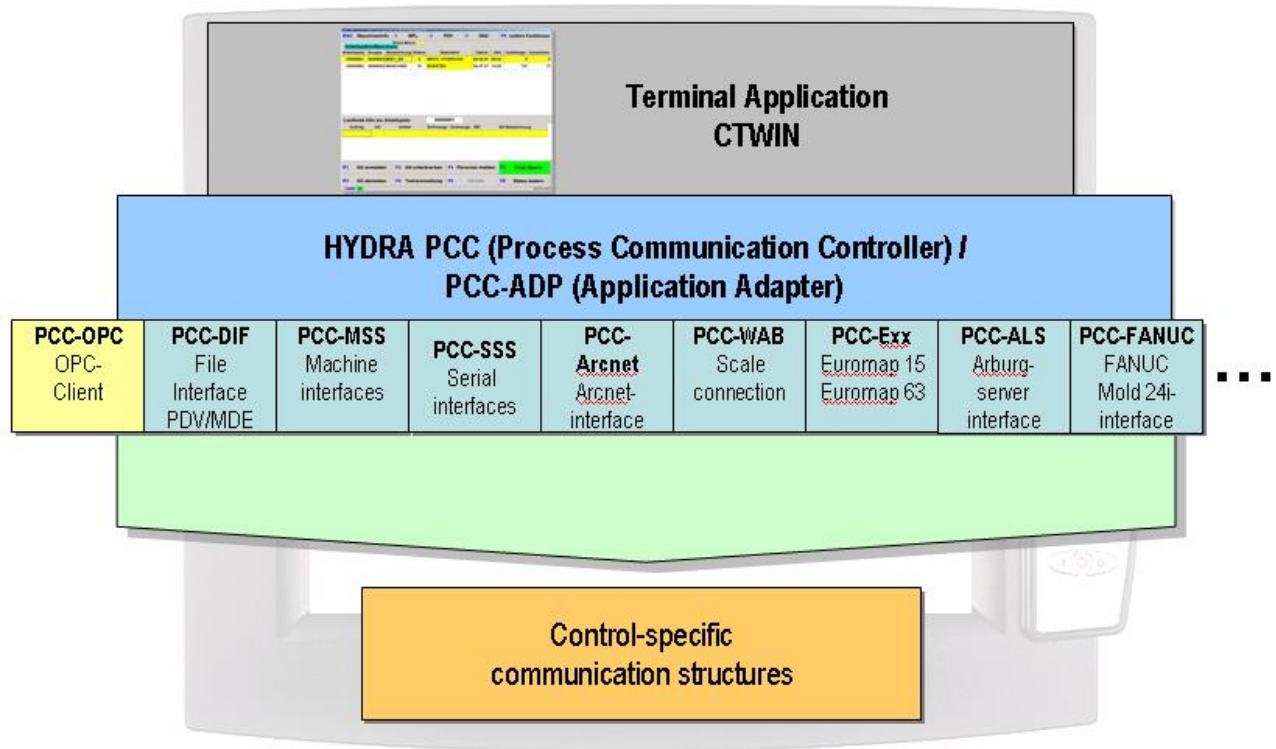


1 PCC Architecture

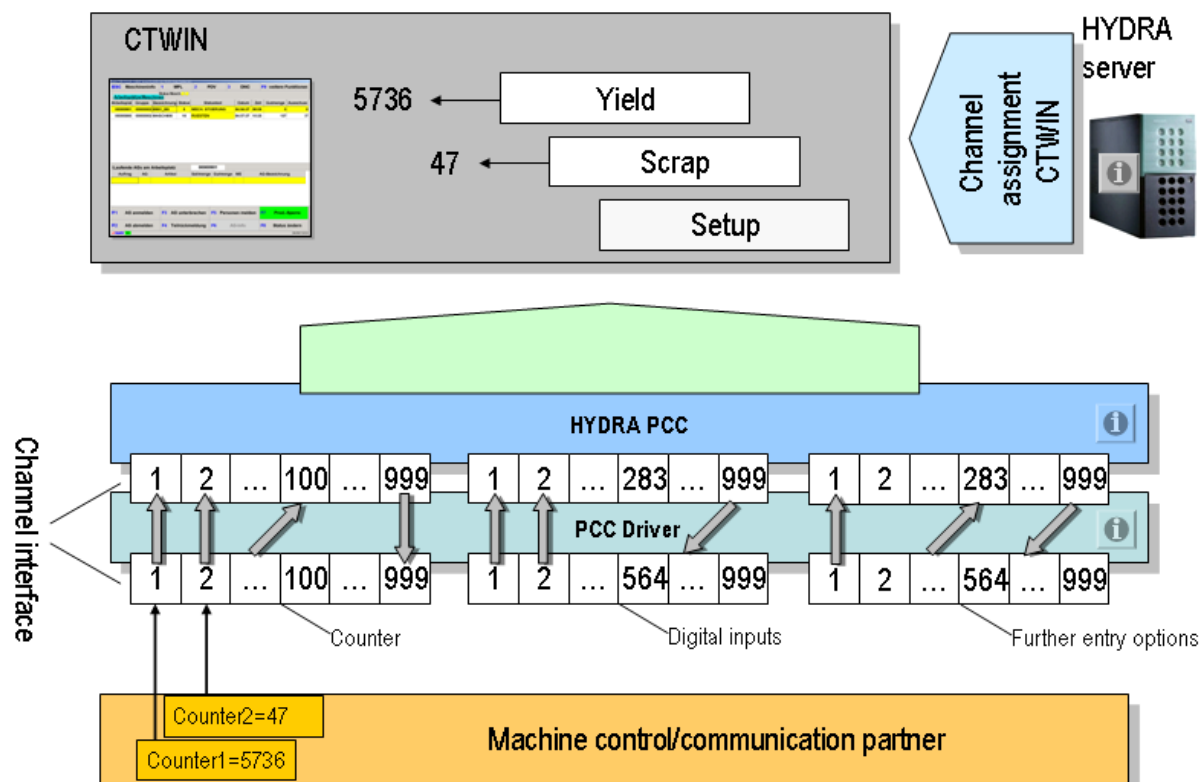
1.1 PCC Modules



The Process Communication Controller (PCC) is a software that contains different modules. For each machine type or each connection type, a separate protocol module is available that can communicate in the relevant 'language' of the connection. Different modules can be used at the same time if the hardware requirements are respected.

PCC-ADP is a configurable extension to map communication processes between machines and the HYDRA shop floor software (e.g. CTAIP terminal software). For this powerful tool, a separate workshop and relevant customization services including documentation are required.

1.2 The channel principle



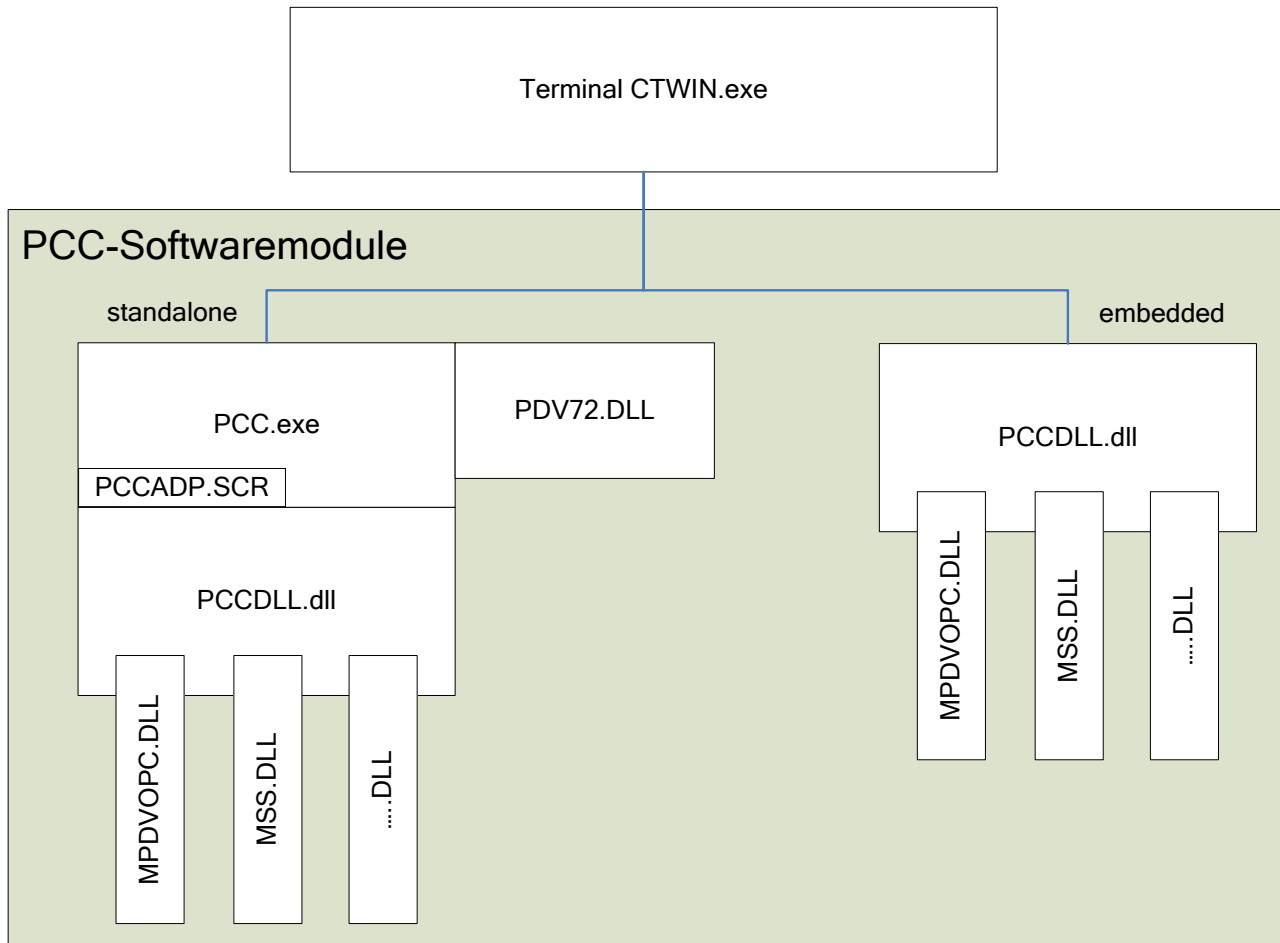
All connections between HYDRA and PCC protocol modules are based on the channel principle. HYDRA addresses logical channels that are represented by physical channels within the PCC protocol module. This PCC-internal process is invisible to HYDRA.

Naming conventions for channels:

Description	Indicator	Max. number	Sample configuration	Module
Counter	C	999	C:C001=	MDE
Digital input	I	999	I:I005=	MDE
Digital output	O	999	O:O004=	MDE
Machine status	M	1 / Machine	M:MSTAT@900=	MDE
Actual cycle times	Z	999	Z:Z010=	MDE
PDV/Measuring channel	P	9999	P:P4537=	PDV
Trigger channel	T	999	T:T029=	PDV
PDV/Event channel	B, F, H	9999	F:F0001=	PDV
DNC channel	D	1 / Machine	D:DNC_M4711=	DNC
Values	V	No restriction	V:EGR_GUT=	ADE, MPL, etc.

Important: Via the PDV/event channel, events in text format or as integer values cannot be recorded. Here, the bit data type or Boolean data type is supported in the form of *true* and *false* (0 and 1).

1.3 Architecture of the software components



The software components are configured and connected using INI files. The following operation modes are available.

1.3.1 Embedded PCC

PCC software and protocol modules are directly integrated in the CTWIN/CTAIP terminal software. The terminal therefore directly integrates the pccdll.dll. This mode is recommended if only the modules HYDRA-MDE, HYDRA-PDV (7.1) and HYDRA-DNC are used (standard versions).

Note: The AIP2 terminal does not support this mode.

Embedded PCC

	can be used	PDV blade	MDE blade	PDV blade MDE blade	MDE blade 2	PDV blade MDE blade 2
CTWIN	✓	✗	✗	✗	✗	✗
CTAIP	✓	✗	✗	✗	✗	✗
AIP 2	✗	✗	✗	✗	✗	✗
PCC without terminal	✗	✗	✗	✗	✗	✗

1.3.2 Stand-alone PCC with AIP2 without PDV

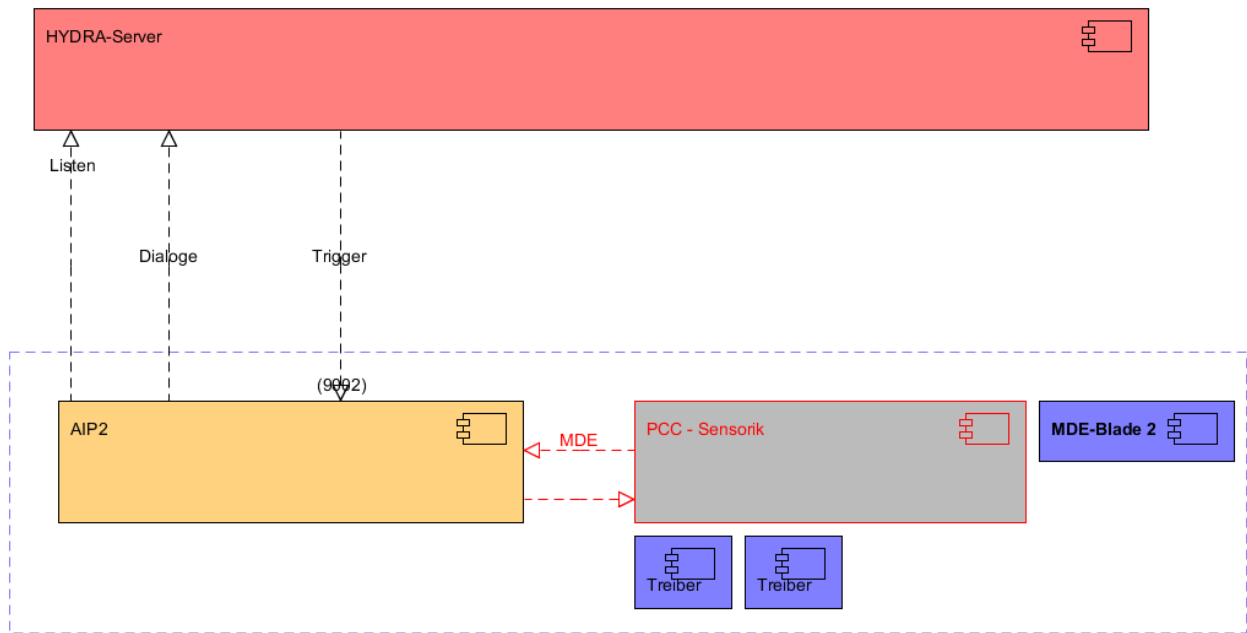
Use case: **AIP2 + MDE without PDV**. With this operation mode, the PCC software is also started as a stand-alone program (pcc.exe) from the AIP2. You need **not create a terminal configuration** for the PCC software. The AIP2 provides the data required for the PCC software. The AIP2 also passes postings of the PCC software to the server. This operation mode is also called AIP2 combined operation.

The AIP2 terminal software automatically starts the PCC software if the option *Operated as MDE terminal* is set in the AIP2 terminal configuration.

Note: You can only use the MDE blade 2 with this scenario.

Stand-alone PCC without terminal configuration

	can be used	PDV blade	MDE blade	PDV blade MDE blade	MDE blade 2	PDV blade MDE blade 2
CTWIN	✗	✗	✗	✗	✗	✗
CTAIP	✗	✗	✗	✗	✗	✗
AIP 2	✓	✗	✗	✗	✓	✗
PCC without terminal	✗	✗	✗	✗	✗	✗



1.3.3 Stand-alone PCC (with PDV)

The PCC software is a stand-alone program (pcc.exe), which communicates with the CTWIN/CTAIP/AIP2 and/or the server via TCP/IP communication. The driver modules and other software are directly integrated in this program via DLL. This mode is required if an extended communication with PCC-ADP is performed or if the PDV 7.2 is used.

This operation mode is also used for custom extensions.

In this mode, you must create a **separate terminal configuration** for the PCC software.

Note for CTWIN/CTAIP: You must not configure the MDE blade 2.

Note for AIP2 and MDE: With this scenario, you must configure the MDE blade 2.

Stand-alone PCC with terminal configuration

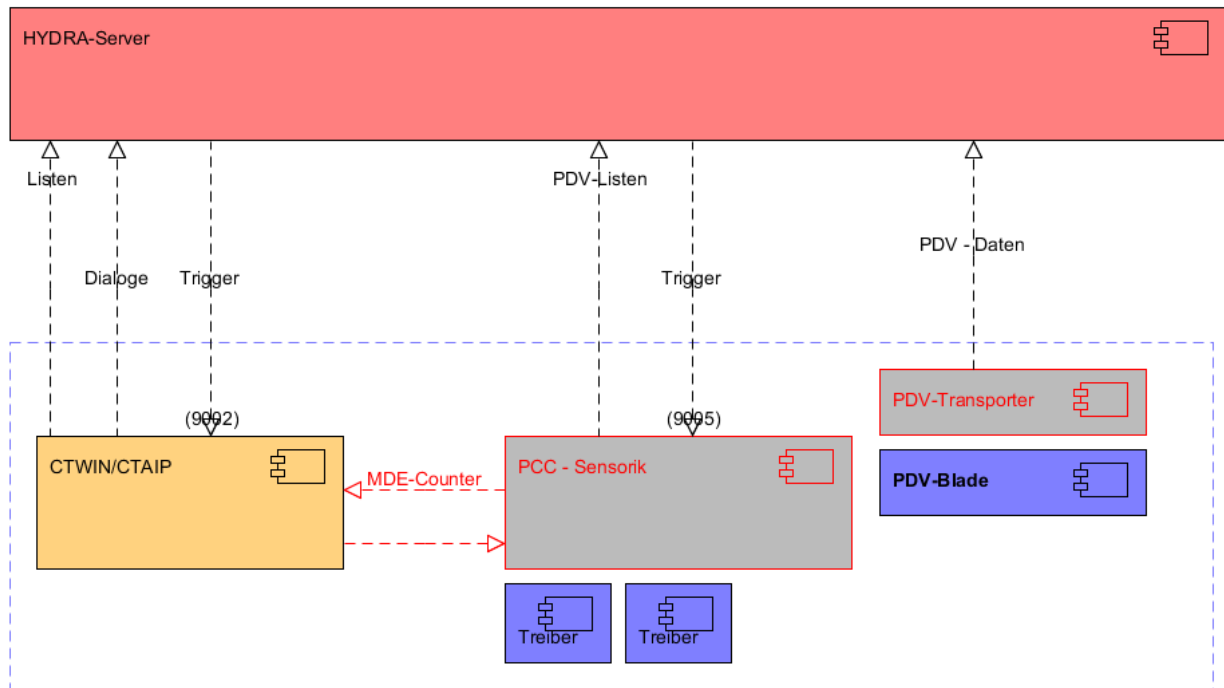
	can be used	PDV blade	MDE blade	PDV blade MDE blade	MDE blade 2	PDV blade MDE blade 2
CTWIN	✓	✓	✗	✗	✗	✗
CTAIP	✓	✓	✗	✗	✗	✗
AIP 2	✓	✓	✗	✗	✓	✓
PCC without terminal	✓	✓	✓*	✓*	✓*	✓*

* Application scenario: stand-alone PCC without terminal software with MDE blade: you must not combine the MDE blades in one installation. You must either use the MDE blade (mdeb.dll) or the MDE blade 2 (mdeb2.dll). Better use the MDE blade 2.

1.3.3.1 Stand-alone PCC (with PDV) with CTWIN/CTAIP

Use case: The CTWIN/CTAIP and the PCC software are run on the same hardware. The PCC software must provide the MDE and PDV data. The MDE counter postings are transferred to the server via CTWIN/CTAIP. The PDV data is transferred to the server via PDV transporter.

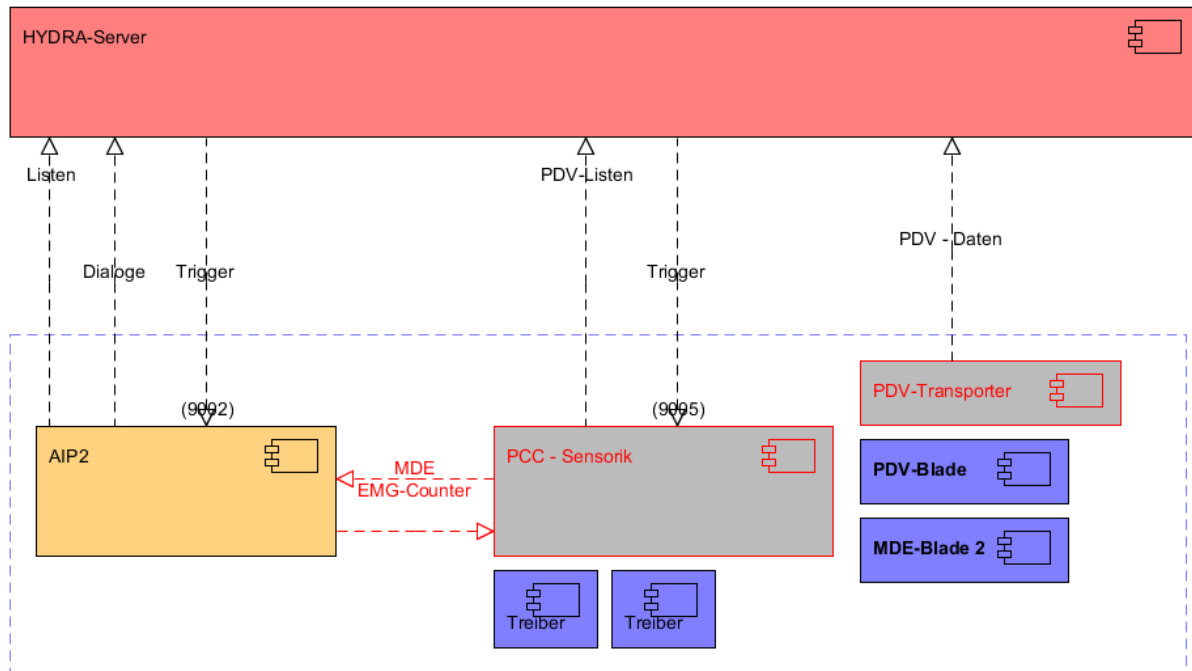
Note: In the PCC, an MDE blade must not be activated. The CTWIN/CTAIP performs the MDE processing.



1.3.3.2 Stand-alone PCC (with PDV) with AIP2

Use case: The AIP2 and the PCC software are run on the same hardware. The PCC software must provide the MDE and PDV data. The MDE data is transferred to the server via CTWIN/CTAIP. The PDV data is transferred to the server via PDV transporter.

Note: The MDE blade 2 performs the MDE processing.



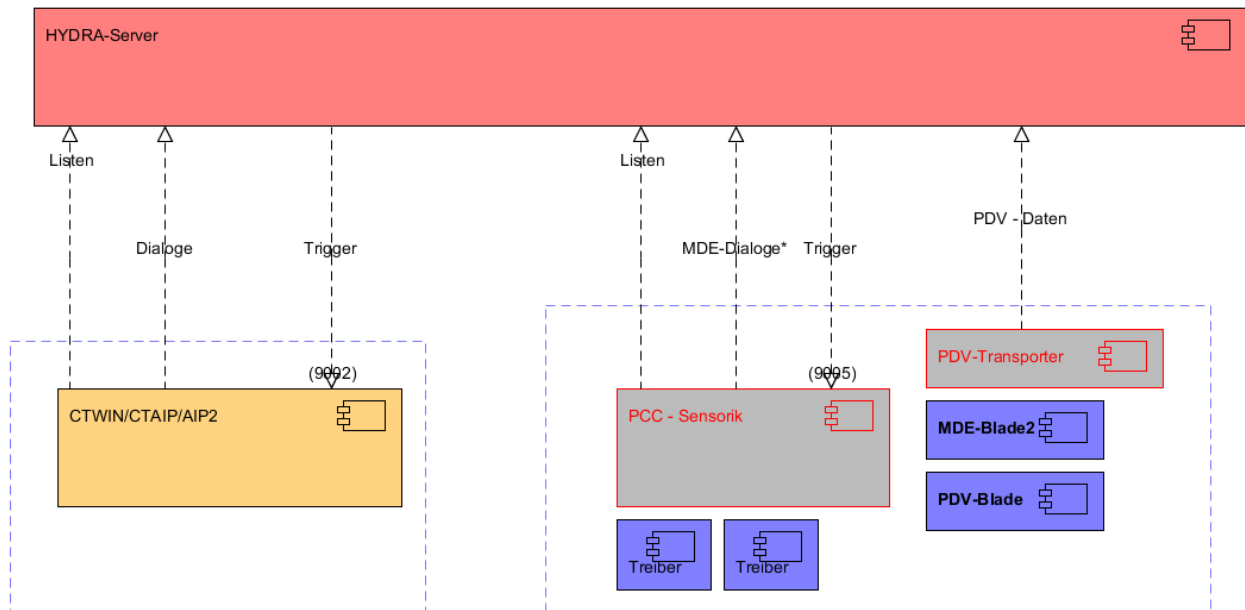
1.3.3.3 Stand-alone PCC without CTWIN/CTAIP/AIP2

Use case: The CTWIN/CTAIP/AIP2 and the PCC software are run on separate hardware.

The PCC can process MDE data and post this data to the HYDRA server.

The PCC can post PDV data to the HYDRA server.

The PCC can post MDE and PDV data to the HYDRA server.



With this operation mode, the MDE dialogs can be transferred to the HYDRA server online or via PDV transporter.

2 Configuration of PCC and components

In general, MPDV performs the implementation process. This document only describes the basic settings and configuration files that are of interest for the customer's system administrator.

2.1 Configuration of the AIP2 terminal (ctaip.ini)

2.1.1 Embedded PCC integration

The embedded PCC is not supported when operated with the AIP2.

2.1.2 Stand-alone PCC integration

Explanations on the parameters (ctaip.ini):

Section	DLL
Ident	Description
BusDLL=PCC.exe	AIP2 starts PCC.exe
BUSSTARTMODE= SYNCHRONIZE You may not set the following configurations: =NONE =ONTIMER	CTAIP start behavior when PCC.exe is started (SYNCHRONIZE) PCC.EXE is started. When the program is started and all DLLs are loaded, the program informs the terminal program AIP2 that the AIP2 can be started. The AIP2 waits for this information. (ctaip.exe then performs all initializations one after the other) This setting must be used with PDV 7.2 / 8.1! (NONE) PCC is not started as separate program but integrated as DLL. This mode cannot be used if the PCC is integrated as executable program. (ONTIMER) PCC.exe is started when the time-controlled main processing of the AIP2 terminal program is executed for the first time.

2.2 Configuration of CTAIP terminal (ctaip.ini)

2.2.1 Embedded PCC integration

Explanations on the parameters (ctaip.ini):

Section	DLL
---------	-----

Ident	Description
BusDLL=PCCDLL.dll	PCCDLL.DLL is directly integrated

2.2.2 Stand-alone PCC integration

Explanations on the parameters (ctaip.ini):

Section	DLL
Ident	Description
BusDLL=PCC.exe	CTAIP starts PCC.exe
BUSSTARTMODE=NONE The following assignments are possible: =SYNCHRONIZE =ONTIMER	CTAIP start behavior when PCC.exe is started (NONE) PCC is not started as separate program but integrated as DLL. This mode cannot be used if the PCC is integrated as executable program. (SYNCHRONIZE) PCC.EXE is started. When the program is started and all DLLs are loaded, the program informs the terminal program CTAIP that the CTAIP can be started. The CTAIP waits for this information. (ctaip then performs all initializations one after the other) This setting must be used with PDV 7.2 / 8.1! If you use AIP2, this setting must also be used. (ONTIMER) PCC.exe is started when the time-controlled main processing of the CTAIP terminal program is executed for the first time.

2.3 Configuration of CTWIN terminal (ctwin.ini)

2.3.1 Embedded PCC integration

Explanations on the parameters (ctwin.ini):

Section	DLL
Ident	Description
BusDLL=PCCDLL.dll	PCCDLL.DLL is directly integrated

2.3.2 Stand-alone PCC integration

Explanations on the parameters (ctwin.ini):

Section	DLL
Ident	Description
BusDLL=PCC.exe	CTWIN starts PCC.exe
BUSSTARTMODE=NONE The following assignments are possible: =SYNCHRONIZE =ONTIMER	CTWIN start behavior when PCC.exe is started (NONE) PCC is not started as separate program but integrated as DLL. This mode cannot be used if the PCC is integrated as executable program. (SYNCHRONIZE) PCC.EXE is started. When the program is started and all DLLs are loaded, the program informs the terminal program CTWIN that the CTWIN can be started. The CTWIN waits for this information. (ctwin then performs all initializations one after the other) This setting must be used with PDV 7.2! (ONTIMER) PCC.exe is started up when the time-controlled main processing of the CTWIN terminal program has been executed for the first time.

2.4 PCC configuration (PCC.ini)

This configuration is not required if PCCDLL is directly integrated.

In combined operation of AIP2 and MDE without PDV, you can copy the pcc.bsp to pcc.ini. No further configuration is required.

Explanations on the parameters (pcc.ini):

Section	SYSTEM
Ident	Description
SYSTRAY_TIME=xx	Time in seconds till the application can be found in the systray
ALIVE_SIGNAL=xx	Every xx seconds, PCC.exe sends a signal to the application to inform that PCC.exe is still running
RESTART=OFF Default=ON	On restart, sends DLG=BUS.ALIVE RESTART=TRUE if one of the parameters is assigned a value

RUNSTANDALONE=ON ON or OFF	PCC.exe runs completely alone and is not started by any application. The user must start the PCC.exe.
TRANSMISSION_RETRIES=2	Number of attempts to transfer data to the terminal program
TRANSMISSION_FAILED= STOP	PCC.exe is stopped when the number of TRANSMISSION_RETRIES is reached.
TRANSMISSION_FAILED=STOP;ctaip.exe Parameters are separated by semicolon	PCC.exe is stopped when the number of TRANSMISSION_RETRIES is reached and when ctaip.exe does not run any longer. (CTAIP has been finished)
TRANSMISSION_FAILED=CONTINUE	PCC.exe is <u>not</u> stopped when the number of TRANSMISSION_RETRIES is reached. The program continues as usual.
QUEUESIZE=1000000	Defines the size of data memory. Specified in bytes
ForceQueueEmpty <i>Available as of pcc.exe > 7.2.2.46</i>	Default = 5 Controls the number of functions executed one after the other to send data Minimum = 1
MainLoopThreadPriority <i>Available as of pcc.exe > 7.2.2.46</i>	Default = tpNormal Timer Thread used to send data (interval 50msec). tpIdle The thread is only executed if the system is in idle process. Windows never interrupts another thread in favor of a thread having the priority level "tpIdle". tpLowest The thread priority is two levels below the normal level. tpLower The thread priority is one level below the normal level. tpNormal The thread has normal priority. tpHigher The thread priority is one level higher than the normal level. tpHighest The thread priority is two levels higher than the normal level. tpTimeCritical The thread has highest priority
NORMALQUEUECYCLE=10000 FASTQUEUECYCLE=1000	Cycle to empty the PCC queue default If more than 10 data records are in the queue, the cycle time to empty the queue is reduced The cycle time is specified in milliseconds
AUTOTERMINATE=xx Specified in seconds	10 seconds are set by default If the PCC.EXE is not closed within this time, the DLLs are killed automatically and the PCC.EXE is closed. To prevent PCC.EXE from getting frozen

Section	Tracing
Ident	Description
TraceLevel=1	Trace level for logging

Section	GateWay Communication
Ident	Description
Port=9003	The port used to communicate with the directly connected CTWIN/CTAIP/AIP2 terminal program via gateway communication to the terminal
TimeOut=3000	TimeOut (by default 3000 msec) to send data to the EVCOM terminal socket (by default 9002). Note: If a PCC-ADP script sends a series of requests, this value must be greater than > 120000 msec to avoid parallel requests at the EVCOM terminal socket. This timeout delays the processing of PCCDLL events.

Section	Server communication
Ident	Description
Active=1 Port=9005	Port that is used for the communication with the event server (EVCOM), normally the HYDRA server. Communication is activated by Active=1. As of version V# 7.2.2.71, the dialogs <SETVAL> and <GETVAL> are exclusively sent to the PCCDLL. All other dialogs are forwarded to the "blades".

Section	WSK
Ident	Description
HOST=10.10.10.1	IP address of the HYDRA server
User=167	User number for the PDM connection to the HYDRA server. (Is identical to the terminal number of the PCC, not the terminal number of the CTWIN/CTAIP!)
TMOUT_M=120	Timeout in seconds for the maximum wait time of a server command (by default 120). When the timeout is exceeded, the terminal goes OFFLINE.

Section	HOST
Ident	Description
BUSSTARTMODE=STAYONTOP the following assignments are possible MINIMIZE STAYONTOP TRAY	Assignments for the start behavior of the PCC.EXE. PCC.EXE is referred to as BUS component. In the [HOST] section, this is the connection to the server of the bus, which is usually the terminal.
BUSHOST=127.0.0.1	The server host address of the bus is entered here – normally the terminal. Default = BUSHOST=127.0.0.1 is set if the entry is not available
BUSTIMEOUT=5	Timeout that is waited for response

	Specified in seconds
BUSPORT=9004	Used port
Active=1 <i>Available as of pcc.exe > 7.2.2.46</i>	Flag to optionally deactivate the communication between PCC and host By default = 1 Does not equal 1 → deactivated

Section	MSS-INIT
Ident	Description
MSS_FILEAGE_MIN	Interval in minutes: counter data saved in this time interval is still sent after a restart of the MDEB2 blade. This configuration is only relevant for the operation "Stand-alone PCC without AIP2".

2.5 PCCDLL configuration (pccdll.ini)

In the PCCDLL.INI file, the module is registered and for each module a separate section is created. The sections are numbered consecutively:

[DRIVER_n]

driver=<protocol module>.DLL

Example:

```
[SERVICE]
tracing=1
ShowErrorWindow=0

[DRIVER_1]
driver=OPCMPDV.DLL

[DRIVER_2]
driver=PCCDIF.DLL
```

You can use the [SERVICE] section to make additional settings, for example for the error logging and for internal purposes.

The names DRIVER_1, DRIVER_2, DRIVER_3, etc. must be used as names for the systems → DRIVER_x
x=system number

IMPORTANT: Conditions and quantities of drivers/DLLs:

		Number of characteristics / Process parameters per machine	Number of machines	Fastest recording cycle
PCC-OPC		Up to 300 or 400	Up to 20	4 seconds (1 second possible, but only for special applications)
PCC-DIF	automatically	Up to 300	Up to 20	-
	cyclically	Up to 300 With cyclic data collection, the number of characteristics is limited when combined with a very fast collection cycle.	Up to 20	5 seconds
FILE-DRVM	automatically	Up to 800	Up to 10	-
WSS driver	cyclically	Up to 200	Up to 4	5 seconds

PDV collection (process data collection):

If PDV values are expected from different drivers, you must set the following parameters in the [SERVICE] section. This function is available as of version pccdll.dll 7.2.1.58.

[SERVICE]

PDV_MULTI_DRVIVER=ON

WAIT_CHAN_COMPLETE=2000

The spelling of the parameter PDV_MULTI_DRVIVER has been corrected as of version 7.2.2.3.

- The wrong spelling is still supported if it is already in use

PDV_MULTI_DRIVER=ON ← correct

PDV_MULTI_DRVIVER=ON enables the function in the PCCDLL.DLL.

WAIT_CHAN_COMPLETE is the timeout (specified in milliseconds). Within this time, all drivers must send their data.

As soon as the timeout is exceeded, all data read so far is sent to the application PDV and the missing channels are added as escalation channels.

PDV_NO_DECIMAL_CONVERSION=ON

The decimal separator is not converted.

By default: PDV_NO_DECIMAL_CONVERSION=OFF.

2.5.1 Notes on the PDV timer processing in the pccdll

Windows timers, which are integrated in the PCC, control the cyclic PDV collection. The number of timers and the interval settings of these timers can affect the performance and lead to a data overflow in the PCC if configured inappropriately. For this reason, the PCC has restrictions for these settings. The number of timers depends on the number of machines and on the number of different collection rules for the machines.

The function is available as of the version 7.2.2.7

You can use the following parameter to deactivate the below procedure:

Entry in the file "pccdll.ini"

[SERVICE]

TIMERCORRECTION=OFF

Note on the deactivation:

Deactivation can lead to an overload situation. Result: the PCC application does not react or the PCC.EXE memory usage increases. If you then close the PCC using the Task Manager, data is lost.

If the user who installs the application, deactivates the timer restriction, this is the user's responsibility.

Procedure, specifications for the automatic timer restriction:

The below inspection of PDV timer intervals is always activated by default.

The PDV timer interval is monitored in pccdll.dll as of version 7.2.1.7.

- 1) if more than 2 PDV timers with an interval ≤ 1 second are configured
→ all timers are set to an interval of 2 seconds
- 2) if more than 5 PDV timers with an interval ≤ 2 seconds are configured
→ all timers are set to an interval of 3 seconds
- 3) If more than 10 PDV timers with an interval ≤ 3 seconds are configured
→ all timers are set to an interval of 5 seconds

The timers themselves are currently not restricted. Furthermore, automatic corrections also result in log outputs - see the below section.

The collection process can be scaled by assigning machines to PCC terminals accordingly.

2.5.2 Trace files, logging of timer changes:

→ \spool\timeerr.log
→ \spool\ pccdll_log.startup.log

The automatic implementation of a timer interval in the pccdll.dll is recorded in the "timererr.log" file.

At the same time, this is also logged in the "pccdll_log.startup.log" file.

A data record is written as follows for each timer of a PDV group:

11-01-26 13:43:08.031; nnnnn To avoid stability problems PDV-Timer Group:AUSP02C1 changed 1000 to 2000 mSec.

If the escalation management function is in use, entries are also escalated using the ERRPRO.ERROR_PROTOCOL_WRITTEN event.

2.5.3 Check of the PDV input channels by the driver

Note: No check takes place with the parameter BYPASS_PDV.

The parameter does not affect this section.

This parameter checks the PDV channels that a driver sends to the application.

It is checked whether a PDV channel sent by a driver is actually active.

[SERVICE]

BYPASS_PDV=ON

CHECK_PDV_INPUT_CHANNEL=ON

By default: all PDV channels that are requested are not checked once again.

By default: **CHECK_PDV_INPUT_CHANNEL=OFF** → No check takes place

Note for BYPASS_PDV=ON:

If P:Pxxx channels are configured in the driver with SETVALEVENTS=, you must not transfer these channels with other IDs as "DLG=GETVAL|V:VXXX=xx|P:PXXX=XX|".

In case of a change, SETVALEVENTS automatically triggers an event for this channel.

If P:Pxxx IDs are included in the data record, the entire data record is sent to HYDRA-PDV as "DLG=R_PMW|...." dialog.

2.5.4 HYDRA-DNC with HYDRA-PDV

Parameters to block PDV driver requests when DNC transfer is active in the driver.

As of version 7.2.1.54

You can use the following entries to block cyclic or trigger-controlled PDV requests in a driver when a DNC transfer is running in the driver.

Use: Driver cannot block PDV requests. Only one communication process with the machine can run at a time.

[SERVICE]

DNC_BLOCKED_PDV=ON ← enables the function

DNC_TIMEOUT=120 ← Timeout in seconds

DNC_TIMEOUT=xx xx in seconds 120 seconds are entered by default

The driver is unblocked after the specified period of time.

If the transfer of DNC data to the machine cannot be finished.

The time must be long enough to ensure that the DNC transfer can be properly completed in any case.

These entries apply to all machines connected to the terminal.

DNC_TIMEOUT=0 ← 0 disables the timeout check

Further parameters for debugging and logging

[SERVICE]

tracing=5 ← logging is enabled (5 = highest log level)

ShowErrorWindow=2 ← debug window is shown for each new data record

2.5.5 Central counter conversion with a factor

Function: The counter is divided by a conversion factor.

The counters can be converted. Create a section [COUNTER_FACTOR]. In this section, configure all counters that must be converted.

You must list the counters here exactly as they are configured in the driver.

{counter ID}={conversion factor}

Only enter a numeric value as factor.

Configuration in the file pccdll.ini

Example:

[SERVICE]

COUNTER_FACTOR=ON → Entry enables conversion

In this section, the counters and the conversion factors are listed

[COUNTER_FACTOR]

C:C001=10

C:C002=3.5

➔ or counter with machine number, example machine 4711

C:C001@4711=10000

All counters that are configured with a conversion factor are additionally logged in the \spool directory in the file "pccdII_CountFaktor.log".



The function of the central counter conversion is only available in the operation mode stand-alone PCC without terminal/AIP.

2.6 Module configuration (pccxxx.ini)

For each protocol module, a separate INI file is available. For information on the configuration, refer to the descriptions of the protocol modules.

3 Configuration of PCC as service

If HYDRA-PCC is operated without the HYDRA shop floor software (stand-alone operation), you can also install HYDRA-PCC as service.

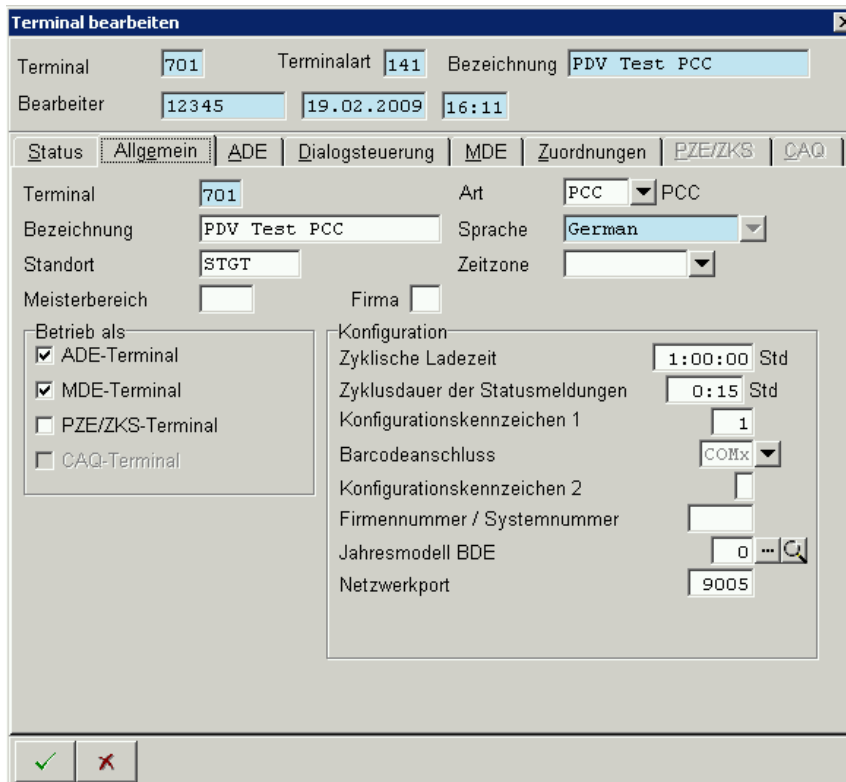
This configuration is useful if you use the PCC as shop floor server. You can install several PCCs as service on one physical computer.

3.1 Preparations: HYDRA console

If you want to install several shop floor servers on one PC, you must ensure that a separate communication port between HYDRA server and PCC.EXE is defined for each system.

The standard port is 9005 and is defined in the terminal configuration.

Edit terminal → General → Network port



Port for system 1: 9005 (default)

Port for system 2: 9006

....

The local configuration is made in the PCC.INI file of the relevant shop floor server via the entry port=xxxx.

PCC.INI:

[Server-Communication]

; COMMUNICATION WITH SERVER VIA EVCOM ; NECESSARY FOR SENDING INFORMATION FROM SERVER TO CLIENT ; MUST BE ACTIVE (= 1) WHEN PDV ; PORTS MUST BE UNIQUE WHEN MULTIPLE SYSTEMS ON SAME PC ARE ACTIVE

Active=1

Port=9005

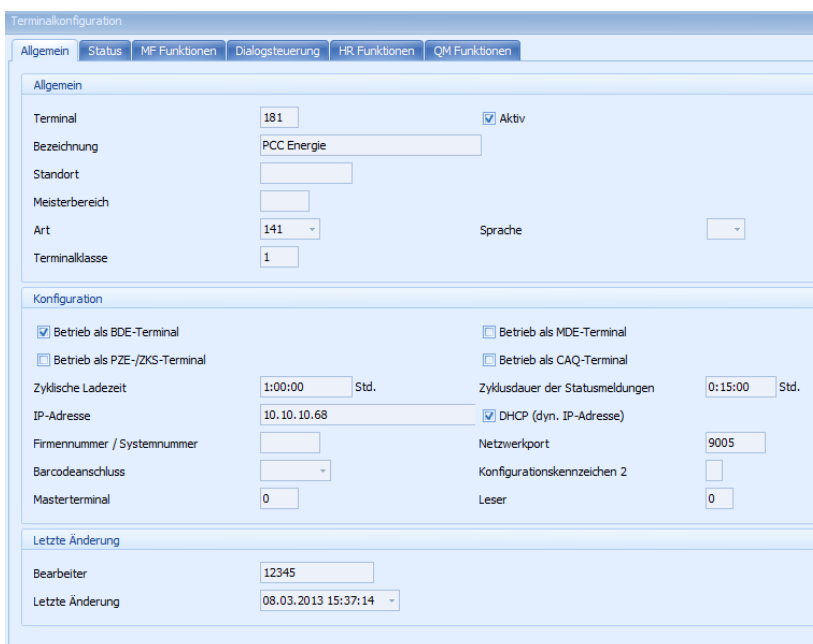
For each system, you must enter the unique port configured on the console.

3.2 Preparations under HYDRA 8

If you want to install several shop floor servers on one PC, you must ensure that a separate communication port between HYDRA server and PCC.EXE is defined for each system.

The standard port is 9005 and is defined in the terminal configuration.

System administration → Terminal → Terminal configuration → General → Network port



Port for system 1: 9005 (default)

Port for system 2: 9006

....

The local configuration is made in the PCC.INI file of the relevant shop floor server via the entry port=xxxx.

PCC.INI:

```
[Server-Communication]

; COMMUNICATION WITH SERVER VIA EVCOM

; NECESSARY FOR SENDING INFORMATION FROM SERVER TO CLIENT

; MUST BE ACTIVE (= 1) WHEN PDV

; PORTS MUST BE UNIQUE WHEN MULTIPLE SYSTEMS ON SAME PC ARE ACTIVE

Active=1

Port=9005
```

For each system, you must enter the unique port configured on the MOC.

3.3 Installation on the shop floor server

- 1) Create a directory x:\hytdisp\ctwin{terminal number} for each PCC where the PCC is then installed.
- 2) Copy the required Windows tools to the PC in the directory:

x:\hytdisp\Service\

The tools are stored on the HYDRA server under

<HYDRADIR>\products\pcc\service

The tools must NOT be deleted!

- 3) In the configuration file pcc.ini, set a separate terminal number each: [WSK]

user={terminal number}

Also deactivate the gateway communication:

```
[GateWay-Communication]
```

```
Active=False
```

- 4) In the configuration file pdv_dll, set a separate terminal number each: (only relevant if you use PDV or if you collect performance data in EMG using process channels)

[Common]

usr={terminal number}

- 5) Start each installed PCC in the foreground and check if the PCC starts and runs properly and without error messages. If a window pops up that you must click, this means that the PCC has a hangup when started as service.
- 6) Install PCC server as service (see also x:\hytdisp\Service\install.bat):

instsrv "PCC Server {terminal number}" x:\hytdisp\Service\SRVANY.EXE

Note:

Call program in the directory "x:\hytdisp\Service\".

- 7) Install PCC transporter as service (see also x:\hytdisp\Service\install.bat):

(only relevant if you use PDV or if you collect performance data in EMG using process channels)

instsrv "PCC Transporter {terminal number}" x:\hytdisp\Service\SRVANY.EXE

Note:

Call the program instsrv in the directory "x:\hytdisp\Service\".

8. Change registry:

Start regedit.exe

and in

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\PCC Server {terminal number}

create the new key "parameters" using the string

Application=x:\hytdisp\ctwin{terminal number}\pcc.exe

Proceed as follows:

To create the new key "parameters": In the window on the left hand side, right-click

"PCC Server XXX"

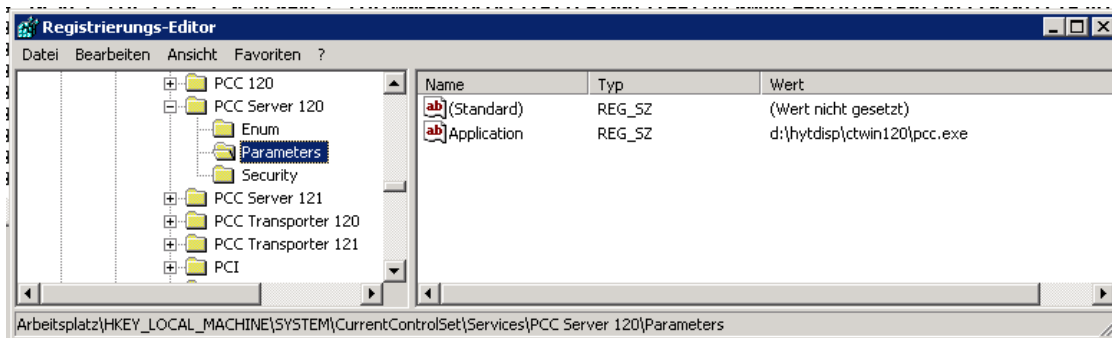
and select in the context menu

New→Key

Right click *Parameters* to open the context menu and select

New→new character string

to create the variable "Application". Enter the path of the relevant PCC program as value.

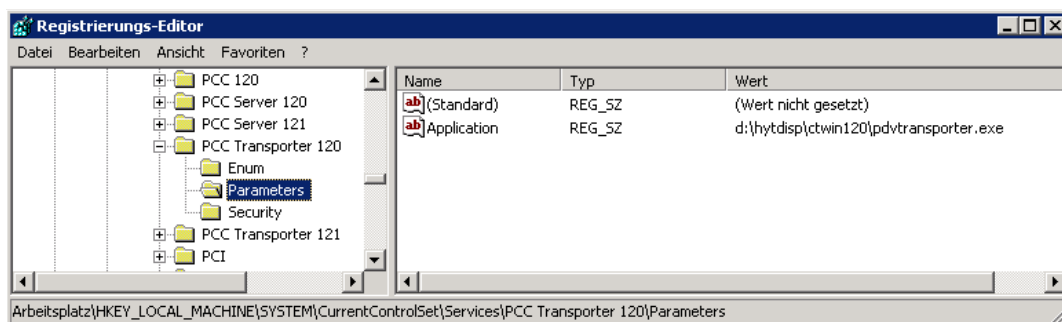


- 9) Repeat the same steps for the transporter: (only relevant if you use PDV or if you collect performance data in EMG using process channels)

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\PCC Transporter {terminal number}"

create the new key "parameters" using the string

Application=x:\hytdisp\ctwin{terminal number}\pdvtransporter.exe

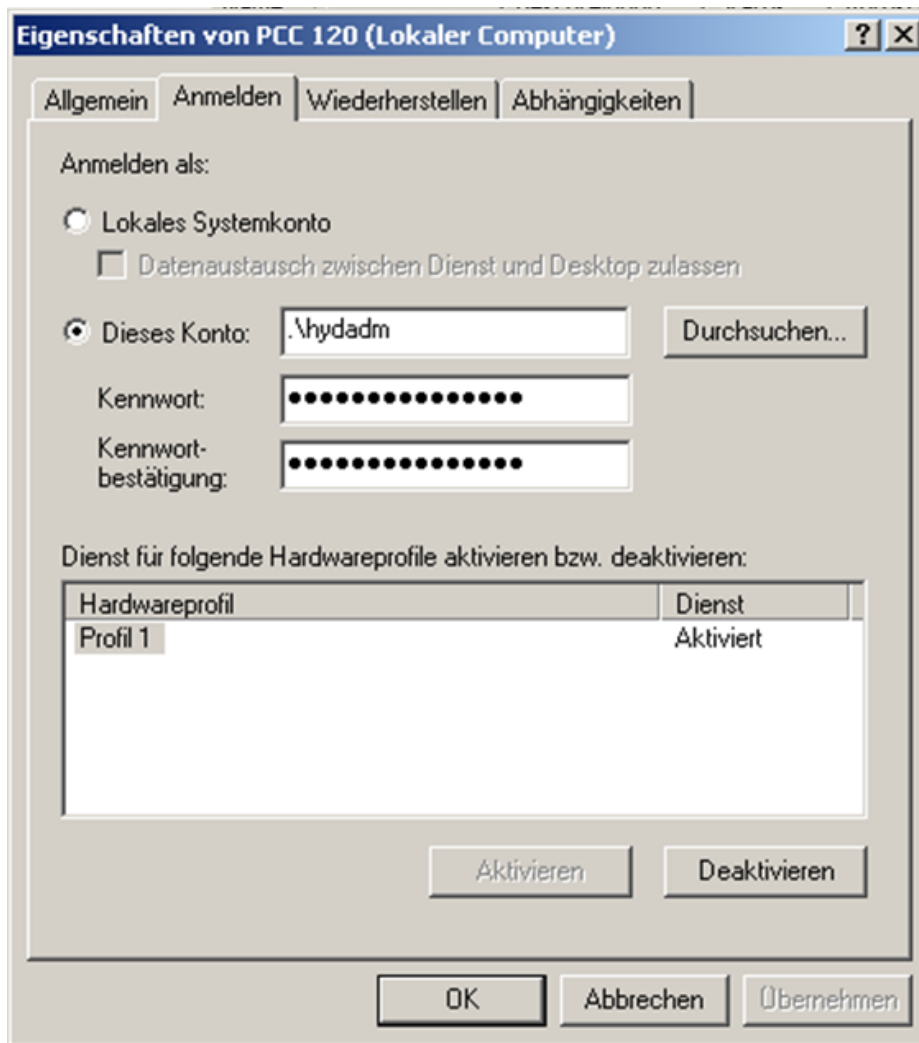


- Configure the
 - PCC server
 - and the PCC transporter (PCC transporter is only relevant if you use PDV or if you collect performance data in EMG using process channels)

in the service control

(Settings→System control→Administration→Services)

and set the start mode *Automatic*. Log on the service in tab *Log on* using the local user account "hydadm".



Notes:

If the PCC program does not start properly, go to the section *Log on* and enable the option *Allow service to interact with desktop* as a test. The start of PCC server or PCC transporter is then visible.

You can identify where the PCC program has a hangup. To identify this, you can also start the service from the Management Console.

You must disable this option at run time!

If you use drivers that require special rights (OPC because of DCOM), it can be useful to run the service using the local system account.

- 10) Other option: You can configure the services so that the service HYDRA Server Agent starts and stops the services at the same time as the HYDRA terminal dispatcher. To configure this, use the service program regedit.exe to add the terminal services in the registry in the key HKEY_LOCAL_MACHINE\SOFTWARE\MPDV\Hydra\Setup in the value Services. This value lists all services that the HYDRA Server Agent starts (one service per row). Add a new row for each terminal service "HYDRA terminal {terminal number}".
- 11) PCC server and PCC transporter can be started via the service control or via the command

```
„net start "PCC Server {terminal number}"
```

```
„net start "PCC Transporter {terminal number}"
```

- 12) To test the configuration, start the PCC services manually before the system restart. Use the service control or the command

```
„net start PCC Server {terminal number}"
```

```
„net start PCC Transporter {terminal number}"
```

The lists read by the server and the different logs must be written in the subdirectories "spool" of the PCC programs.

The PCC programs are stopped via the service control or the command

```
„net stop PCC Server {terminal number}"
```

```
„net stop PCC Transporter {terminal number}"
```

If the service "HYDRA Server Agent" is used to start the terminal programs, this service must also be stopped (paused) because otherwise it would restart the terminal program.

Notes:

- 1) A service that is not properly logged on, can be logged off using

```
sc delete "PCC Server {terminal number}"
```

or

```
sc delete "PCC Transporter {terminal number}"
```

Example:

sc delete "PCC Server 120"

2) Here, it is not possible to download the PCC program from the server to update the version because the HYDRA tool "inst32.exe" does not support a download to different directories. You must manually copy a new PCC program into the separate directories or a distribution software of the customer's must be used.

3.4 Service does not start OPC server

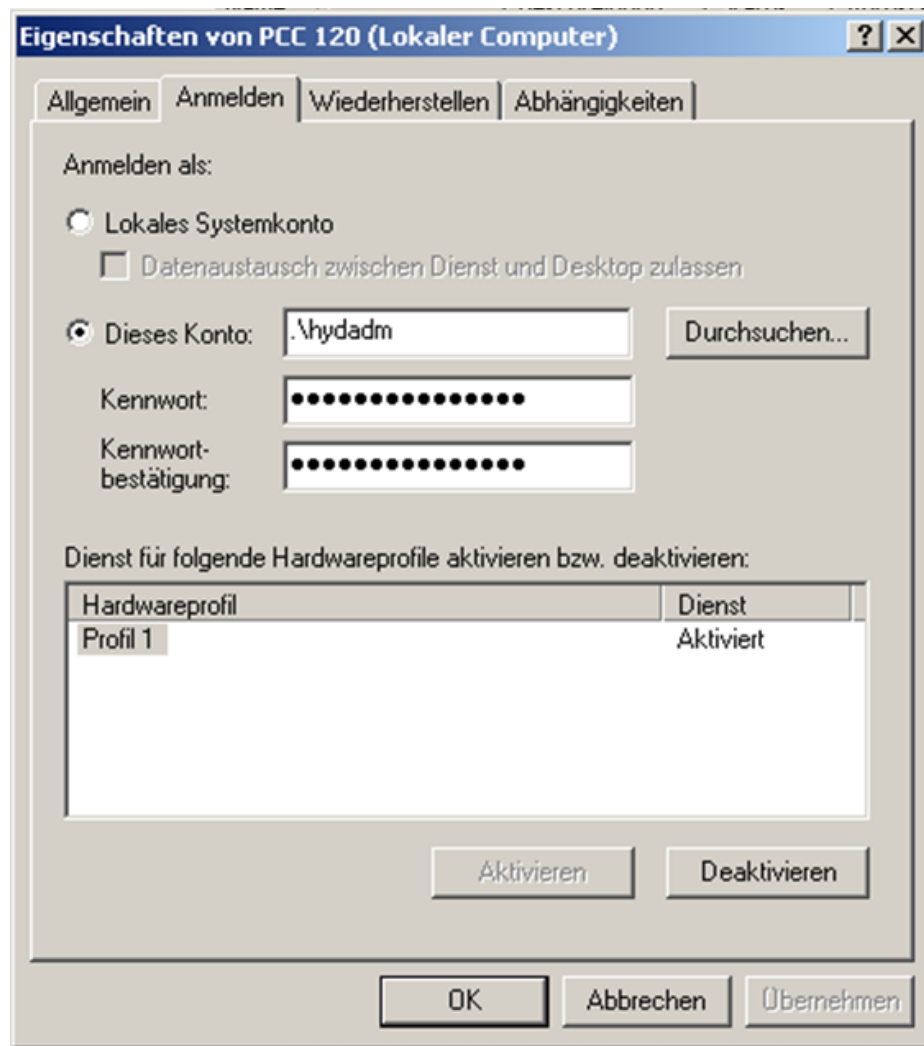
Problem: The service does not start the OPC server.

Solutions: If you enable the setting "Allow service to interact with desktop", the service can properly start the OPC server, the data is again recorded.

Service on local shop floor PC







Diagnosis

For diagnosis purposes or as part of the PCC configuration and implementation of a machine connection, you can start a PCC service in the foreground as follows:

Open the service control

Start → Settings → System control → Administration → Services

Select the service "PCC server {terminal number}"

{terminal number} = terminal number configured for the terminal

Note down the start mode configured in tab *General*.

In tab *General*, change the start mode to *Deactivated* and take over the changes using the button *Apply*. Close the service using the button *Cancel*.

The PCC can now be started in the foreground in the directory `x:\hytdisp\ctwin{terminal number}` using the `pcc.exe` .

{terminal number} = terminal number configured for the terminal

x = installation directory of the HYDRA services on the HYDRA terminal server

To run the PCC again as service, proceed as follows:

Close the PCC, which runs in the foreground, using the shortcut ALT-F4 and confirm the Close dialog.

Open the service control

Start → Settings → System control → Administration → Services

Select the relevant service "PCC server {terminal number}"

{terminal number} = terminal number configured for the terminal

In tab *General*, change the start mode to the mode previously noted down and take over the changes using the button *Apply*. The service is automatically restarted within a minute via the service HYDRA Server Agent.

4 Application blades

If the PCC is run in stand-alone operation, you can implement application functionality for the PCC. This application functionality is integrated in a so-called blade DLL. The PCC can then perform application-specific operations. One example is the Process Data Processing as of version 7.2. Independent of the terminal software, the PDV blade performs the recording of the PDV information and sends this information to the server in an own data channel. And the PDV blade acts as online data server for the display of process data in the network. In the future, further blades will be able to provide the functions of an application.

The blades are integrated in the PCC.ini file in the [BLADES] section:

```
[BLADES]
; THESE BLADES HAVE TO BE LOADED
BLADE_1=.\\blades\\MDEB.DLL
BLADE_2=.\\blades\\PDV.DLL
...
```

Note:

If you integrate a blade, you must also check the configuration INI file of the blade and create this file, if required.

This section describes how to configure these blades or refers to manuals where further information can be found.

4.1 PDV blade configurations **pdv.dll**

Configurations are stored in the pdv_dll.ini file.

The following table describes the INI options of the file `pdv_dll.ini` that controls the processing of the `pdv.dll`. For the `pdv_dll.ini` file, the sample file `pdv_dll.bsp` is delivered by default, which is copied and changed accordingly.

For the implementation, only set the parameters below. Use the default values for the other parameters:

[Common]	
Usr=701	← PCC terminal number
[Transport]	
IP=192.168.10.75	← Server, port and file port for file transport
Port=10377	
FilePort=10303	
[Blade]	
VisualOnline=Y	← Activation of the data server for the online visualization
AutostartTransport=Y	← Automatic start of the transporter.

Parameter overview of PDV DLL.ini:

Section	Common
Ident	Description
Transportpath	Path where data files are stored. Do not select the spool directory of the ctwin/ctaip terminal because this directory can be emptied at any time. Example: Transportpath=pdv/transport/ Consequently, files are stored in the path ctwin/pdv/transport/ IMPORTANT: The folder structures must exist and, if required, created manually before starting the program!
Transportappendix	File extension to transport mass data (measured values). By default pmas - do not change.
Instantappendix	Extension for files including limit violations or PDV events. By default pitt - do not change
Esk appendix	File extension for escalation messages. By default pesk - do not change
Usr	Terminal number of the PCC terminal as specified on the console. Must be identical to the entry [WSK] ->User of the pcc.ini. Is required for server communication
InstantMode	You can specify two different modes for the transport of limit violations. This configuration can have a significant impact on the system performance. The following two modes are supported: <i>Single</i> Limit violations are buffered in one single file and inserted in batch processing, once a specific file size/number of data records has been reached. <i>Multi</i> Each dialog of a limit violation is written into a separate file. The file is immediately ready for transport. This mode is outdated and not recommended. Always select the <i>Single</i> mode. IMPORTANT: Only change this entry after consulting MPDV because the system functionality cannot be guaranteed otherwise.

Section	Blade
Ident	Description
Handlecount	Number of open entries in the data file (mass data) until it is closed
Filesize	Maximum file size of data files of mass data. This value is specified in relation to the customer's data volume (large data volume --> large Filesize). Specified in bytes (i.e. 1000 = 1KB)
MaxDataRuns	Specifies the number of times the PDV sender has to run before data files are generated. If the value is greater than 0, an open data file is closed automatically after n runs - even if no data is available. For this reason, "open" files are also transported even if data is no longer recorded. This mechanism is disabled if the value is 0.

MaxWaitTime	You can define a time after which a file is completed in any case. Time in seconds. This function is disabled if the value = 0 or a wrong value is specified.
PDVTracelevel	Current trace level of <code>pdv.dll</code> IMPORTANT: For performance reasons, tracing must always have the value 1 in live operation, because another value greatly affects the performance and correct functioning of the system cannot be guaranteed.
PDVTraceMaxLines	Maximum number of data rows in the trace file
PDVTraceMaxSize	Maximum data size of the trace file
Monitoring	Specifies if an output is active or not. Default = N - do not change
Transport	Type/technology to transport data Only <code>FILE</code> is currently supported and must not be changed
VisualOnline	Currently not available, must be set to N
MNRDebug	Debug mode replacing the machine number by a consecutive counter. Must not be set to Y in live operation! Y = Active N = Not active
AutostartTransport	Specifies if the transport program is also started when <code>pdv.dll</code> is started (by the <code>pcc.exe</code>). Y = Active N = Not active Should be set to Y in live operation
HardDiscQueue	Specifies if a hard disk queue is used to pass measured values. Valid values are Y = Yes / yes with hard disk queue N = No / no without It is recommended to work without hard disk queue because this greatly increases the performance. Only activate the hard disk queue if the system stability has absolute priority and if the data volume is low!
CallESK	Controls if within the PDV an escalation message is sent to the HYDRA escalation management when a limit value is violated. Valid values: Y = Yes, ESK is active N = No, ESK messages are not sent By default, the escalation is activated.
ESKWaitTime	Time in seconds that must pass at least between two escalations triggered by limit violations of the same characteristic. Only when the time has passed, a new escalation is triggered. Valid values are between 0 (always new escalations) and 100000. By default: 600 (10 minutes) This configuration is important if the processes show important variations. It helps to avoid a great number of escalations.
EventTimerIntervall	At the time interval specified, a thread timer searches for expired alert status-

	es of configured PDV events. The interval is specified in seconds. If the value is set to 0, no additional timer thread is generated. This way, the performance can be optimized with customers that do not have events that trigger alerts. Otherwise, all values that are greater than 0 are valid (specified in seconds). Example: If a PDV event is configured, which triggers an alert for 10 seconds and the value of the <code>EventTimerIntervall</code> is set to 3, it is checked every three seconds whether the alert has been triggered and whether it must be finished.
BlockEventChannel-Info	Configuration Y or N. Y is set, if event channels are used that are actually sent as event by the data collection function. With the setting Y, the channels are not kept and managed in the internal channel list. The channels are no longer communicated to the protocol module, as the module sends events independently.
CaptureMillisecondsActive	Configuration Y or N. If the parameter is not specified, the default value N is used. Available as of <code>pdv_dll</code> version 7.2.1.43 If Y is configured, milliseconds are added to the time stamp of the data collection. As a result, the collected process parameters are saved with a time stamp that includes milliseconds in the HYDRA database.

Section	<i>BckLastValues</i>
Ident	Description
TimeInterval	The last values are backed up at the time interval in seconds specified. If the value = 0 the backup is disabled
Backuptime	Not used at the moment. Only the value 0 is valid.
Backupfile	File name used to back up the last values. Example <code>Backupfile=pdv_lv.bck</code>

Section	<i>BckConfig</i>
Ident	Description
TimeInterval	Time interval in seconds that is used to back up the current configuration. If the value = 0 the backup is disabled
Backupfile	File name used to back up the current configuration Example <code>Backupfile=pdv_cfg.bck</code>

Section	<i>DLL</i>
Ident	Description
Filter	Name of the DLL that filters data. Without path specification. The DLL is expected in the "blades" subdirectory. By default: <code>Filter=CompressDll.dll</code>

Sender	Name of the sender DLL. Without path specification. The DLL is expected in the "blades" subdirectory. By default: <code>Sender=PDVSenderDll.dll</code>
Visual	Name of the data server DLL for online visualization. It is expected in the ctwin/aip directory. By default: <code>Visual=dataserver.dll</code>

Section	Receiver
Ident	Description
inpath	Input path for lists relevant to PDV. By default: <code>inpath=./pdv/incoming/</code>

In addition, a transport program is required to transfer data from the shop floor component to the server. Just as it is the case for `pdv.dll`, this is only available for Windows terminals and is called `pdvTransporter.exe`. The program is stored in the same directory as the `pcc.exe`. To start the program, double-click it. If configured in the INI file, the program starts automatically. Only one system of the process may be active at a time (run).

The program is configured as follows:

It uses the same INI file as `pdv.dll` (`pdv_dll.ini`), which must be stored in the same directory as `pcc.exe`. The sections in the following describe the relevant fields of the INI file.

Section	Common
Ident	Description
Transportpath	Relative path that specifies the folder where the files for the transport to the server are stored. You must not use the "ptf" extension!
Transportappendix	File extension used to store mass data files. Must be different to <code>Instantappendix</code> and <code>Backupappendix</code> . You must not use the "ptf" extension!
Instantappendix	File extension used to store data that must be transported immediately. Must be different to <code>Transportappendix</code> and <code>Backupappendix</code>
Usr	Terminal number. Has to match the INI file from <code>pcc.exe</code> and the terminal configuration on the HYDRA server.
InstantMode	Select the mode according to the data volume of <code>pdv.dll</code> , but the <code>pdvTransporter</code> also requires the necessary information how it must transport the files. Only make changes to the configurations after consulting MPDV!

Section	Transport
Ident	Description

HoldBackTime	Time in seconds that specifies how long transported files are locally saved.
MaxLines	Maximum number of data records that are buffered while tracing until the handle is closed.
Tracelevel	Trace level of the transporter
Timer	The transport program searches for data in the transport folder at the specified time interval in seconds.
IP	IP address of the server process <code>hypdvsvr.exe / out</code>
Port	Network port number of the server process <code>hypdvsvr.exe / out</code>
FilePort	Port number of the HYDRA file server
Backupappendix	File extension used to save the backed up files. Must be different to <code>Transportappendix</code> and <code>Instantappendix</code> You must not use the "ptf" extension!
ClearBackup	Defines the number of times the transport program has to run before locally "expired" files are deleted. The value should not be too low, as otherwise the performance might be affected. The default value is 500.

4.2 Configurations of the MDE Blade `mdeb.dll`

4.2.1 General configurations

Configurations are filed in the `mdeb.ini` file.

The file must be stored in the application directory of the PCC.EXE program; a sample file `mdeb.bsp` is delivered

There are two sections [SERVICE] and [INIT]

[SERVICE]

→ Configuration of the log file level Default=1

Log file: `.\spool\mdeb.dll.text`

LOGLEVEL=0

[INIT]

→ An escalation message is not sent to the HYDRA server.

ESCALATIONSEND=OFF

→ Interval setting to cyclically transfer the counters with `M_AST` data records to the server

Setting is made in seconds Default = 120

MDECOMMINTVL=90

→ Interval to rename the MDE file for the PDVTransporter for transports to the server

Setting is made in seconds Default=10

Transportintvl=10

Parameter CYCLE_AST_INTVL=xxx**as of mdeb.dll version 7.2.1.21 12.06.2012**

If the parameter is not set, a default value of 900 seconds is set (15 minutes).

Function: If no data record DLG=M_AST (including quantities) is generated for the relevant machine within this period, a data record (DLG=M_AST|...) without quantities is sent to the server.

The data record is required because only then the system can make a possible shift change for the machine on the server.

Example of a data record generated without quantities :

DLG=M_AST|MNR=FUELL02||DAT=06/12/2012|ZEI=68240|DLGMODE=AUTO|USR=2120|OFF=J|

CYCLE_AST_INTVL=xxx ← specified in seconds

Note: The additional cyclic request of the interval is made in the interval settings using the parameter MDECOMMINTVL

Within the cycle of (MDECOMMINTVL) it is only checked if the time of CYCLE_AST_INTVL has expired.

Important: special case: Parameter is set to 0

CYCLE_AST_INTVL=0

If the parameter is set to 0, data records with DLG=M_AST|... with or without quantities are generated with the interval setting using the parameter MDECOMMINTVL.

→Interval to request the outputs to be set

Is included as of mdeb.dll version 7.2.1.13**This requires the program pcc.exe as of version 7.2.2.71.**

From this version on, the server is requested to set the outputs for machines in the driver, when mdeb.dll is restarted.

You can use the parameter below to change the interval used to set the outputs. At this interval, the server requests the outputs from the driver and writes them. The entry is specified in seconds.

By default, 900 seconds are set (every 15 minutes).

You can change the setting using the following entry.

You make the configuration in the file mdeb.ini.

[INIT]**MDEINITINTVL=900****MDEINITINTVL=0 ← Completely disables initialization**

Configuration of outbound IDs within the OPC client opcmpdv.ini

Outputs are configured as follows

The ID always starts with V:Vxxx

xxx --> is always 3 characters long and identifies an output channel from 001 to 999

Example for channel 1, machine 4711 and channel 10 for machine 4712

V:V001@4711=the OPC item for the output channel that is set

V:V010@4712=the OPC item for the output channel that is set

Outputs are always set with signal 1 (ON)

Outputs are always reset with signal 0 (OFF)

4.2.3 Mode to convert nominal output, set output and actual cycle

The following settings are available using this parameter.

The configuration refers to the collected values M_SZY, M:EZY, Z:Zxxx(IZY values)

Section [INIT]

CYCLE_MODE=1 units per hour 3600000 / machine value

CYCLE_MODE=2 units per minute 60000 / machine value

CYCLE_MODE=3 seconds per cycle machine value * 1000

If no parameter is entered, the default setting is 0.

The collected value is transferred to the server without being converted.

4.2.4 DIGIN channel processing (extension for I:IXXX channel configuration)

This configuration is available as of mdeb.dll version 7.2.1.28.

The cycle time processing is not made in the MDE blade. This is only a status change.

For the recording of input signals

These input signals must send the status 0 or 1.

Signal 0: The signal is not available. (OFF)

Signal 1: The signal is available. (ON)

IDs used for the digital input in the driver:

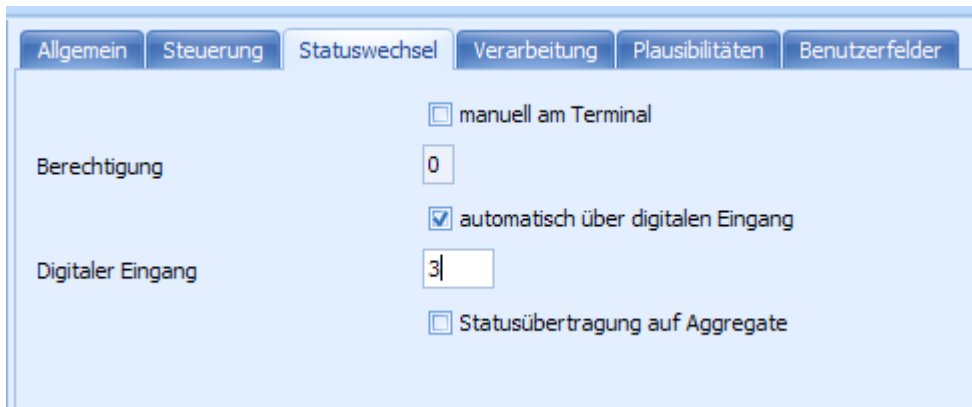
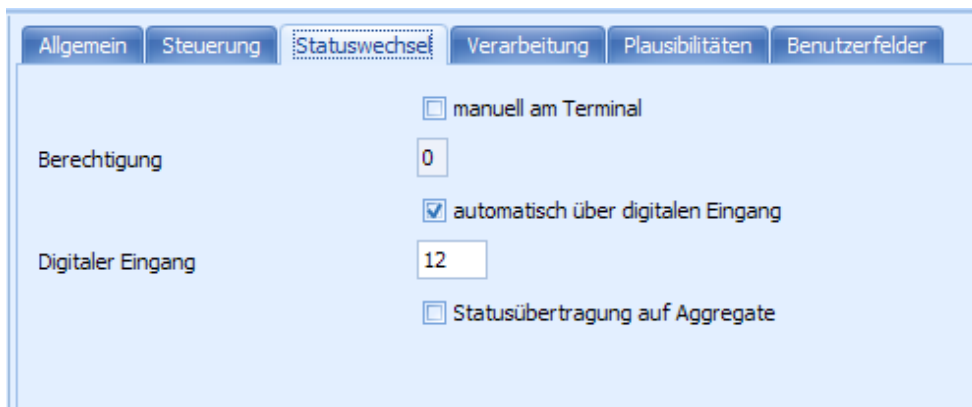
Configuration: I:IXXX (for XXX, 001 to 999 is possible)

Configuration for a status in the database

For the relevant status, a digital input [] must be configured.

If this input is available (set to 1), the respective status is sent to the server.

Note: Status *Production* takes priority.

Configuration in the driver for a digital input

Example: driver **opcmpdv.dll**

Configuration in the file **opcmpdv.ini** for the digital inputs 3 and 12

I:I003={here the OPC item that sends the signal for this input}

I:I012={here the OPC item that sends the signal for this input}

Note: Important!

You must assign unique digital input numbers for a terminal.

You must not assign a digital input number several times on a terminal.

The numbers must be consecutive numbers.

Example of machine statuses on the terminal.

Machines 4711, 4712, 4713 are assigned to the terminal.

All machines have the same status.

The digital inputs must have a consecutive configuration.

Procedure to change status of the inputs

The input for the status with production identifier "P" takes priority. If this input is set, no other set input is processed.

If the input for the status with production identifier "P" is reset, the next set input with the smallest number is used and send to the server as status.

The input for status production identifier "P" is not set, another status is set. If the input of a set status is reset, the next set input channel with the smallest number is sent to the server.

Channel is reset: Channel is set to 0.

Channel is set: Channel is set to 1.

Example: Machine configuration for a terminal

Machine	Status	Digital input The configuration must be unambiguous. You must not assign a number several times.
4711	1	1
	2	2
	3	3
4712	1	4
	2	5
	3	6
4713	1	7
	2	8
	3	9
	4	10

	5	11
--	---	----

4.3 Configurations of the MDE Blade2 for the AIP2 (mdeb2.dll)

4.3.1 General configurations

Configurations are filed in the mdeb2.ini file.

The file must be stored in the application directory of the PCC.EXE program; a sample file mdeb2.bsp is delivered

There are two sections [SERVICE] and [INIT]

Section	SERVICE
Ident	Description
LOGLEVEL	Setting of the log file level. Default=1 0 - Logging is disabled 5 - troubleshooting 9 - highest setting for the troubleshooting Log file:.\spool\mdeb2.dll.text Example: LOGLEVEL=0

In section [DS100_TEXT], the displayed texts for the DS100 can be overridden. The text may only include ASCII characters. All umlauts and Unicode characters are not permitted. The sample values listed below are the default values.

Section	DS100_TEXT
Ident	Description
YIELD:	Example: YIELD=GUT:
SCRAP	Example: SCRAP=AUS:
USR_INPUT_STATUS	The DS100 device interprets the hashtag as placeholder for an input character. Example: USR_INPUT_STATUS=Status:####
DOWNTIME	Example: DOWNTIME=Stillstand
STATUS_NOT_AVAILABLE	Example: STATUS_NOT_AVAILABLE=Nicht vorhanden

NO_SHIFT	Example: NO_SHIFT=Keine Schicht
PRODUCTION_LOCK	Example: PRODUCTION_LOCK=PSP

Section	<i>INIT</i>
Ident	Description
ESCALATIONSEND	No escalation message is sent to the HYDRA server. The default value is ON. Example: ESCALATIONSEND=OFF
MDECOMMINTVL	Interval setting for the cyclic upload of the counters with M_AST data records to the server. The setting is specified in seconds. The default value is 120 seconds. Example: MDECOMMINTVL=90
CYCLE_AST_INTVL	If the parameter is not set, a default value of 900 seconds is set (15 minutes). If no data record DLG=M_AST (including quantities) is generated for the relevant machine within this period, a data record (DLG=M_AST ...) without quantities is sent to the server. Example of a data record generated without quantities : DLG=M_AST MNR=FUELL02 DAT=06/12/2012 ZEI=68240 DLGMode=AUTO USR=2120 OFF=J Note: The additional cyclic request of the interval is made in the interval settings using the parameter MDECOMMINTVL Within the cycle of (MDECOMMINTVL) it is only checked if the time of CYCLE_AST_INTVL has expired. If the parameter is set to 0, data records with DLG=M_AST ... with or without quantities are generated with the interval setting using the parameter MDECOMMINTVL. Example: CYCLE_AST_INTVL=120
IDENT_WITH_MNR	Channel processing with machine number. The default value is OFF. Configuration of machine-related counters in the OPCMPDV driver: C:C001@M1000 = < OPC variable > C:C001@M1001 = < OPC variable > Example: IDENT_WITH_MNR=ON
CYCLE_MODE	Mode for the conversion of nominal capacity, configured capacity and actual cycle. The setting applies for the collected values M_SZY, M:EZY, Z:Zxxx(IZY values).

	The following settings are available using this parameter. CYCLE_MODE=0 no conversion CYCLE_MODE=1 units per hour 3600000 / value of machine CYCLE_MODE=2 units per minute 60000 / value of machine CYCLE_MODE=3 seconds per cycle value of machine * 1000 If no parameter is entered, the default value is 0. The recorded value is passed to the server without conversion. Example: CYCLE_MODE=3
MAX_COUNTER_VALUE	If the difference between two recorded counter values is greater than the value specified here, then an entry is written in the dialog error log on the server. If required, the system can also trigger an escalation. The default value is 2000. Example: MAX_COUNTER_VALUE=500
TRANSPORT	In the AIP2 combined operation with or without PDV, the parameter must be set to ONLINE. All MDE postings of the machines must be posted to the HYDRA server via the AIP2. You can only use the BATCH parameter in combination with the PDV and a stand-alone PCC without AIP2 connection. You must create a terminal configuration for this PCC. To process the data files of a PCC service, you must install an additional service on the HYDRA server (see section Installation of the server service "FILE-DD-Server" for the PCC). Available settings: ONLINE → Online transfer BATCH → File transfer via PDV transporter Example: TRANSPORT=ONLINE
TRANSPORTINTVL	Interval to rename the MDE file for the PDV transporter. Then the transport to the server is performed. In case of a combined operation with AIP2, this parameter is not supported. You can only use this mode in combination with the PDV and a stand-alone PCC without AIP2 connection. The default value is 10 seconds. Example: TRANSPORTINTVL=20
BLADE_TRANSPORTPATH	Using this parameter, you can overwrite the pdv_dll.ini → Transportpath.

Section	NOTIFY#TNR
Ident	Description

HOST	You need not set this value. In the combined operation with AIP2, the AIP2 passes the correct parameter to the PCC. Example: HOST=127.0.0.1
PORT	You need not set this value. In the combined operation with AIP2, the AIP2 passes the correct parameter to the PCC. Example: PORT=9004
TIMEOUT	Send timeout to the AIP2 in milliseconds. The default value is 1000 ms. Example: TIMEOUT=2000
NOTIFY.CYCLE	Update of the AIP2 GUI in seconds The default value is 1 second. Example: NOTIFY.CYCLE=3

The section [customizing options] is intended for customer-specific settings.

Section	<i>Customizing options</i>
Ident	Description
TRGEN_INTERVAL=..	If part quantities are cyclically uploaded, you can regularly update the order data in the higher-level PPS system. Using this setting, the cyclic generation of an upload of part quantities is enabled for all machines of the terminal. (Default = 0 = disabled) Example: Cycle for all machines of the terminal TRGEN_INTERVAL=120 (specified in seconds) When the interval is over, the server processing is triggered when the ID ".. TRGEN=J .." is added to the next cyclic M_AST posting.
TRGEN_INTERVAL@<MNR>=..	Using this setting, you can make a machine-specific configuration. Ex.#1: no generation for machine <MDE001> TRGEN_INTERVAL@MDE001=0 Ex.#2: alternative cycle for machine <MDE002> TRGEN_INTERVAL@MDE002=240

4.3.2 DIGIN channel processing (extension for I:IXXX channel configuration)

Recording of input signals

These input signals must send the status 0 or 1.

Signal 0: The signal is not available. (OFF)

Signal 1: The signal is available. (ON)

IDs used for the digital input in the driver:

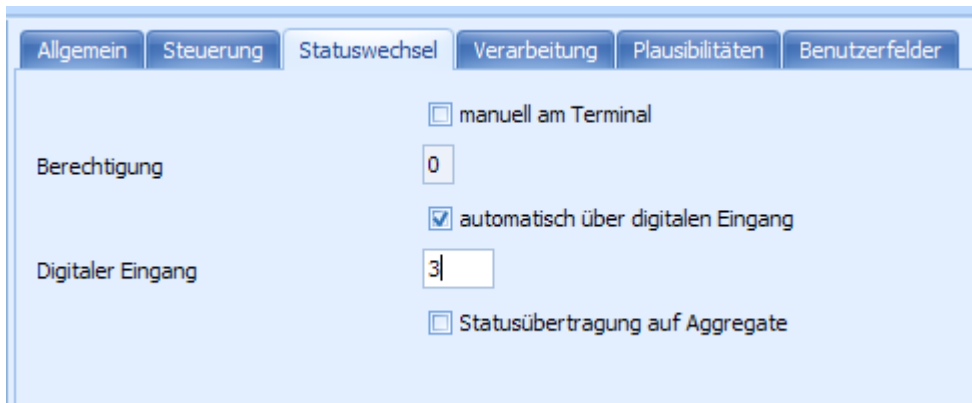
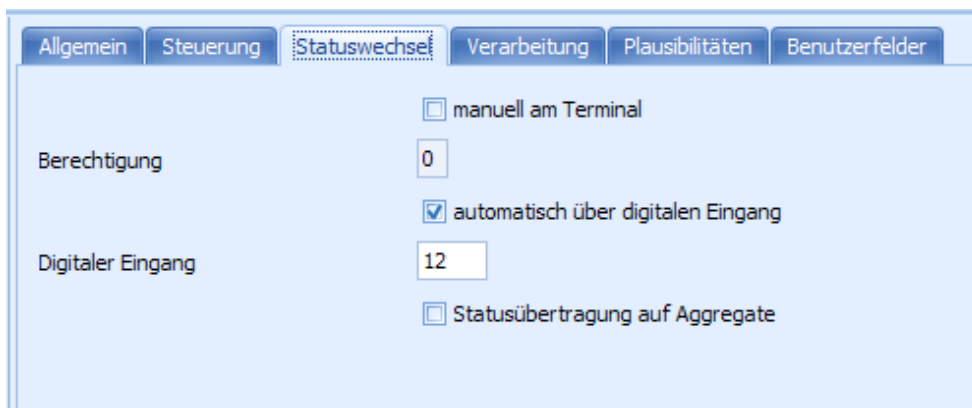
Configuration: I:XXXX (for XXX, 001 to 999 is possible)

MOC configuration for a status.

For the relevant status, a digital input [] must be configured.

If this input is available (set to 1), the respective status is sent to the server.

Note: Status Production takes priority.

Configuration in the driver for a digital input

Example: driver **opcmpdv.dll**

Configuration in the file **opcmpdv.ini** for the digital inputs 3 and 12

I:I003={here the OPC item that sends the signal for this input}

I:I012={here the OPC item that sends the signal for this input}

Note: Input number

You must assign unique digital input numbers for a terminal.

You must not assign a digital input number several times on a terminal.

The numbers must be consecutive numbers.

Example of machine statuses on the terminal.

Machines 4711, 4712, 4713 are assigned to the terminal.

All machines have the same status.

The digital inputs must have a consecutive configuration.

Procedure to change status of the inputs

The input for the status with production identifier "P" takes priority. If this input is set, no other set input is processed.

If the input for the status with production identifier "P" is reset, the next set input with the smallest number is used and send to the server as status.

The input for status production identifier "P" is not set, another status is set. If the input of a set status is reset, the next set input channel with the smallest number is sent to the server.

Channel is reset: Channel is set to 0.

Channel is set: Channel is set to 1.

Example: Machine configuration for a terminal

Machine	Status	Digital input The configuration must be unambiguous. You must not assign a number several times.
4711	1	1
	2	2
	3	3
4712	1	4
	2	5
	3	6
4713	1	7
	2	8

	3	9
	4	10
	5	11

5 PCC error handling

The PCC or its integrated protocol modules process the communication channels. If an error occurs, the application is informed. Here, integration and equipment of the PCC decide on the behavior. In case of a stand-alone PCC with terminal communication or in case of an embedded configuration, the application is the terminal. The terminal receives the messages on channel errors. The terminal can then display the errors or forward them to the server where the errors are logged or send as escalations. For details on the processing on the terminals, refer to the terminal manual or the MDE escalation manual (MDE-ESK).

5.1 Escalations of channel errors

All required channels (MDE, PDV, DNC ...) are initialized when the PCC is restarted. If the assignment in the PCC connection is missing for a channel, the escalation `ERRPRO.ERROR_PROTOCOL_WRITTEN` is sent to the application (see the HYD-ESK documentation). An escalation is also triggered if a channel sends an error during operation. The PCC error codes are entered in the `ERRPRO.ERRCODE` field (see the HYD-RET document, error codes as of 4000). "PCC" is always entered in the field `ERRPRO.EREIG` and "SYS" in the field `ERRPRO.ERRCLASS`. The `ERRPRO.BEM` field shows the description of the error including the channel number. If available, the connected machine number is entered in the `ERRPRO.MNR` field, the operation number is optionally entered in the `ERRPRO.ANR` field.

If the terminal is the application, the escalation is processed there, and optionally sent to the server.

If the PCC is used without terminal, an MDE blade DLL can be used to communicate the error as an escalation directly from the PCC to the server.

In the server, the error messages are entered in the OFFLINE dialog log and the `ERRPRO.ERROR_PROTOCOL_WRITTEN` escalation is triggered additionally.

Deactivation of escalations:

Escalations are sent to the server using the MDE blade (`mdeb.dll`) and can be disabled in the `mdeb.ini` file.

In this case, escalations are no longer sent to the server.

See the following parameter in the `mdeb.ini` file in section **[INIT]**

This file is in the application directory of the `pcc.exe`

Settings in `mdeb.ini`

[INIT]

→ No escalation message is sent to the HYDRA server.

ESCALATIONSEND=OFF

5.2 Diagnosis upload

As of PCC in version 7.2.2.78, an upload function of diagnosis files is integrated in the PCC, just as it is the case for the terminal programs CTWIN/CTAIP and AIP2.

The upload is supported for the below scenarios if WSK server communication is enabled:

1. Upload for EXCEPTION
2. Upload by USER action (click button "Upload" in the debugging user interface)
3. Upload via REMOTE request
(Terminal label → request diagnosis files)
 - 3.1 Performing upload using the MOC client
 - 3.2 Performing upload using console clients

The upload is performed into the server directory (identical procedure as for terminal programs)

„< Server>\<Dir>\<system>\spool\spl<User>\upload<User>.pcc.zip“

e.g. Server = win2008-7
 Hydra directory = hydra4
 System = 4
 User = 2120 (→ terminal number + 2000)
results in "\\win2008-7\hydra4\4\spool\spl2120\upload2120.pcc.zip“

If WSK communication is not enabled, the upload is performed via the "leading" terminal (CTWIN/CTAIP/AIP2).

"< Server>\<Dir>\<system>\spool\spl<User>\upload<User>.zip“

e.g. Server = win2008-7
 Hydra directory = hydra4
 System = 4
 User = 2121 (→ leading terminal number + 2000)
results in "\\win2008-7\hydra4\4\spool\spl2121\upload2121.zip“

The below entry is added to the "EXCEPTION" log of the terminal ("./spool/ExcMagic.log") to document the process that triggers the upload.

```
...
12-10-09 10:08:01.631; 5484 b>=== NOTIFY - EXCEPTION - BY - d:\kunden\mpdv\ctwin\pcc.exe ===== ..
12-10-09 10:08:01.631; 5484 ===== see further information in archive @ file[ ./spool/upload.pcc.zip
@ ./spool/madExcept.bugreport.pcc.exe.txt ]
12-10-09 10:08:01.631; 5484 e<=== NOTIFY - EXCEPTION - BY - d:\kunden\mpdv\ctwin\pcc.exe =====..
...
```

In both cases, you can find further information on the reason for the "EXCEPTION" or the application error (with or without WSK communication) in the file "./spool/madExcept.bugreport.pcc.exe.txt".

6 Installation of the server service "FILE-DD-Server" for the PCC

To process the data files of a PCC service, you must install an additional service on the HYDRA server.

Note: If several PCCs are used, you must install one FILE-DD-Server per PCC. For each PCC, you must also configure a separate path where the PCC transporter stores "its" files and where the relevant service can read the files.

You must specify the target path for the PDV transporter in the pdv_dll.ini in the section [Common]:

Example:

```
Otherappendix1=mdat
Otherprefix1=pcc778/MDE
```

In this example, the files are stored on the HYDRA server in the directory <HYDRA-DIR>\<MDT>\spool\pcc778.

6.1 Windows

To install the FILE-DD-Server, the file <system>\hymap.cfg is extended as follows (here: system 1 of the HYDRA system):

```
[HYDRA1 FILE-DD-Server 1]
HY_USR=string,8888
program=%HYDRADIR%\hymw.exe
Fehlerprotokoll=string,%HYDRADIR%\1\err\hymw.fil.err
File=string,*.mdat
```

The changed configuration is activated as follows (here: system 1 of a HYDRA system):

1. Close HYDRA
Close HYDRA Manager
2. In a DOS Shell
cd %HYDRADIR%
ntinst -if 1\hymap.cfg
3. Start HYDRA

Optional parameter for the FILE-DD-Server

- **Path**

Path specification where the service can find the files of the shop floor server that must be read.

The default path is spool (<system>/spool)

Example

```
Path=string,d:\hydra1\1\spool\pcc778
```

6.2 Linux

To install the **FILE-DD-Server**, the file <system>\hymap.dat is extended as follows (here: system 1 of the HYDRA system):

```
;### HYDRA File-DD-Server  
hymw.out -E./1/err/hymw.fl.err -F*.mdat:8888:0:0:0:HYDRA1 File-DD-Server 1
```

To activate the changed configuration, you must restart HYDRA.

Optional parameter for the FILE-DD-Server

- **Path**

Path specification where the service can find the files of the shop floor server that must be read.

The default path is spool (<system>/spool)

```
;### HYDRA File-DD-Server
```

```
hymw.out -E./1/err/hymw.fl.err /PATH=/u1/hydra1/1/spool/pcc778
```

```
-F*.mdat:8888:0:0:0:HYDRA1 File-DD-Server 1
```